설비 고장을 예측하는 AI 데이터 분석

함수 설계와 활용

기본값 인자 → 지역변수와 범위 → 함수 설계와 조합

posco

CONTENTS

목차

01 기본값 인자

02 지역변수와 범위

03 함수 설계와 조합

posco

posco

01

기본값 인자

기본값이 필요한 이유 · 정의 방법 · 순서 규칙 · 덮어쓰기

같은 인자를 반복해 넣는 번거로움

대부분 같은 값을 넣는 매개변수에 매번 같은 인자를 적는 번거로움

함수를 부를 때마다 거의 항상 같은 값을
일일이 적어야 하는 번거로움이 있다.

점검 기록 함수의 단위가 거의 항상 **"도"**라면, 호출마다 "도"를 적는 것은 시간 낭비이고
반복해 적다 보면 **오타나 누락 같은 실수**가 끼어든다.

거의 항상 같은 값을 매번 적는 문제

호출마다 똑같은 값을 적으면 시간이 낭비되고 오타나 누락 같은 실수가 끼어든다.

01 · 시간 낭비

거의 변하지 않는 값을 호출마다 반복 입력

report 압축기A 75.3 **도**,
report 펌프1 88.0 **도**처럼
호출할 때마다 "도"를 반복해 적게 된다.

02 · 실수 유발

매번 적는 만큼 틀릴 기회도 늘어남

같은 값을 반복해 적다 보면
어느 순간 **오타**를 내거나
빠뜨릴 수 있다.

03 · 우리의 바람

자주 쓰는 값은 미리 정하고 다를 때만 변경

거의 항상 쓰는 값은
미리 정해 두고,
다를 때만 바꿔 주는 것.

기본값 인자가 필요한 이유

기본값 인자는 매개변수에 미리 정해 둔 값. 인자를 안 주면 기본값, 주면 그 값이 쓰인다.

인자를 안 주면

미리 정해 둔 기본값이 자동으로 쓰인다

함수를 정의할 때
"이 값은 따로 안 주면 이걸로 한다"는 값을
매개변수에 미리 지정해 둔다.

인자를 주면

그 값이 덮어쓰며 대신 쓰인다

필요할 때는 다른 값을 넣어
기본값을 덮어쓸 수 있어
유연함이 그대로 유지된다.

기본값 인자의 두 가지 가치

기본값 인자의 가치는 편리함, 안전함, 그리고 유연함 유지 세 가지다.

01 · 편리함

코드가 짧아지고 함수 호출이 쉬워진다

자주 쓰는 값을 매번 적지 않아도 되니 코드가 깔끔해진다.

02 · 안전함

오타와 누락이 줄어든다

같은 값을 반복해 적다 생기는 실수가 줄어든다.

03 · 유연함 유지

필요할 때는 다른 값으로 덮어쓰기

print의 줄바꿈도 기본값 동작. 원하면 다른 값으로 바꿔 쓸 수 있다.

posco

기본값 인자 정의 방법

매개변수 뒤에 =값을 적으면 기본값이 정해진다.

CODE

```
def report(name, value, unit="도"):
    print(name + ": " + str(value) + unit)
report("압축기A", 75.3)          # unit 생략 → 기본값 "도"
report("펌프1", 7.2, "bar")     # "bar"로 덮어쓰기
```

unit="도"가 기본값 지정. 따로 안 주면 "도"를 쓰고, 주면 그 값이 쓰인다.

posco

필수 매개변수와 선택적 매개변수

기본값이 있느냐 없느냐로 매개변수는 필수와 선택으로 나뉜다.

01 · 필수 인자

기본값 없음
반드시 인자를 받음

name과 value가 필수 매개변수.
이들을 생략하면 **에러**가 난다.

02 · 선택 인자

기본값 있음
인자 생략 가능

unit이 선택 매개변수.
안 받으면 기본값을 쓰고, 받으면 받은 값을 쓴다.

기본값은 문자열·숫자 등 함수 성격에 맞는 가장 흔히 쓰는 값으로 정한다

기본값은 함수를 쓰기 편하게

자주 쓰는 설정은 기본값으로 묻어 두기.

꼭 필요한 값만 넣어 부르면 된다.

기본값 인자를 잘 쓰면 함수가 한결 쓰기 편해진다.
자주 쓰는 설정은 기본값으로 묻어 두고, 꼭 필요한 값만 넣어 부르면 되니까.

무엇을 기본값으로 둘지 고민하는 것 자체가 **좋은 함수 설계의 일부**이다.

기본값이 있는 매개변수의 순서 규칙

기본값 없는 매개변수가 앞, 기본값 있는 매개변수가 뒤. 어기면 정의하는 순간 에러.

정의	가능	이유
`def f(a, b, c=0)`	가능	기본값 없는 것 앞
`def f(a, b=0, c=0)`	가능	기본값 있는 것 뒤
`def f(a=0, b, c)`	불가	기본값 있는 것 앞

꼭 받아야 하는 값을 먼저, 생략해도 되는 값을 나중에. Python이 헷갈리지 않도록 한다.

필수가 앞 선택이 뒤

기본값 없는 매개변수가 앞, 있는 매개변수가 뒤. 어기면 호출이 아니라 정의 순간 에러.

01 · 호출하는 사람 입장

앞쪽은 반드시, 뒤쪽은 필요할 때만

앞쪽 필수 인자는 반드시 넣고
뒤쪽 선택 인자는 필요할 때만 넣으면 되니
자연스럽다.

02 · 에러가 나는 시점

호출이 아닌 정의하는 순간

에러 메시지를 보고
순서를 바로잡으면 된다.

posco

기본값을 덮어쓰기

기본값이 있는 매개변수에 인자를 넣으면 기본값은 무시되고 넣은 값이 쓰인다.

CODE

```
def grade(temp, limit=80):
    if temp > limit:
        return "점검필요"
    return "정상"

grade(95, limit=90) # 키워드로 limit=90 → 기본값 80 대신 90 사용
```

limit의 기본값은 80. 호출 시 limit=90을 넣으면 90으로 덮어쓴다.

위치로도 키워드로도 덮어쓰기

기본값은 고정된 게 아니라 평소 값일 뿐. 그대로 두면 기본값, 인자를 넣으면 덮어쓴다.

01 · 위치 인자

마지막 자리에 값을 넣어 덮어쓰기

기본값 매개변수가 하나라면
위치 인자로 마지막 자리에
값을 넣으면 된다.

02 · 키워드 인자

이름을 지정해
원하는 것만 덮어쓰기

limit과 label 중 label만 바꾸고 싶다면
키워드로 **label="위험"**처럼
콕 집어 바꿀 수 있다.

실습 1. 기본값 인자 함수 만들기

기본값이 있는 매개변수를 만들고, 생략하면 기본값·넣으면 덮어쓰기를 확인

단계

① def 괄호 안 매개변수에 =로 기본값을 지정
② 인자를 생략하고 호출해 기본값이 쓰이는지 확인
③ 인자를 넣어 호출해 기본값을 덮어쓰는지 확인
④ 필수 매개변수는 앞, 기본값 매개변수는 뒤 순서 규칙 확인

예상 결과 78 → 정상 / 95 → 경고 / (limit 덮어쓰면) 50 → 경고

posco

실습 1. 기본값 인자 함수 만들기 정답

CODE · 정답

```
def status(temp, limit=90):
    if temp > limit:
        print("경고")
    else:
        print("정상")
status(78)            # 정상 (limit 기본값 90)
status(95)            # 경고
status(50, limit=40)  # 경고 (기본값 덮어쓰기)
```

posco

02

지역변수와 범위

함수 안과 밖 변수 · 값의 수명 · 같은 이름 다른 변수

posco

함수 안에서 만든 변수

함수 안에서 만든 변수는 그 함수 안에서만 쓸 수 있다. 이것이 지역변수.

지역변수 LOCAL VARIABLE

함수 안에서 만들어져
그 함수 안에서만 쓰이는 변수

함수 안에서 **result**라는 변수에 결과를 담았다면
함수 안에서는 자유롭게 쓸 수 있지만, 함수 밖에서 출력하려 하면
"그런 변수를 모른다"며 에러가 난다.

불편이 아니라 안전장치

함수 안 변수가 밖으로 새어 나가지 않아 바깥에 영향을 주지 않는다.

01 · 격리

새어 나가지 않는다

함수 안 변수가 밖으로
새어 나가지 않아
바깥에 영향을 주지 않는다.

02 · 정리

끝나면 깨끗이 정리

함수는 자기 안에서
변수를 만들어 쓰고
끝나면 깨끗이 정리한다.

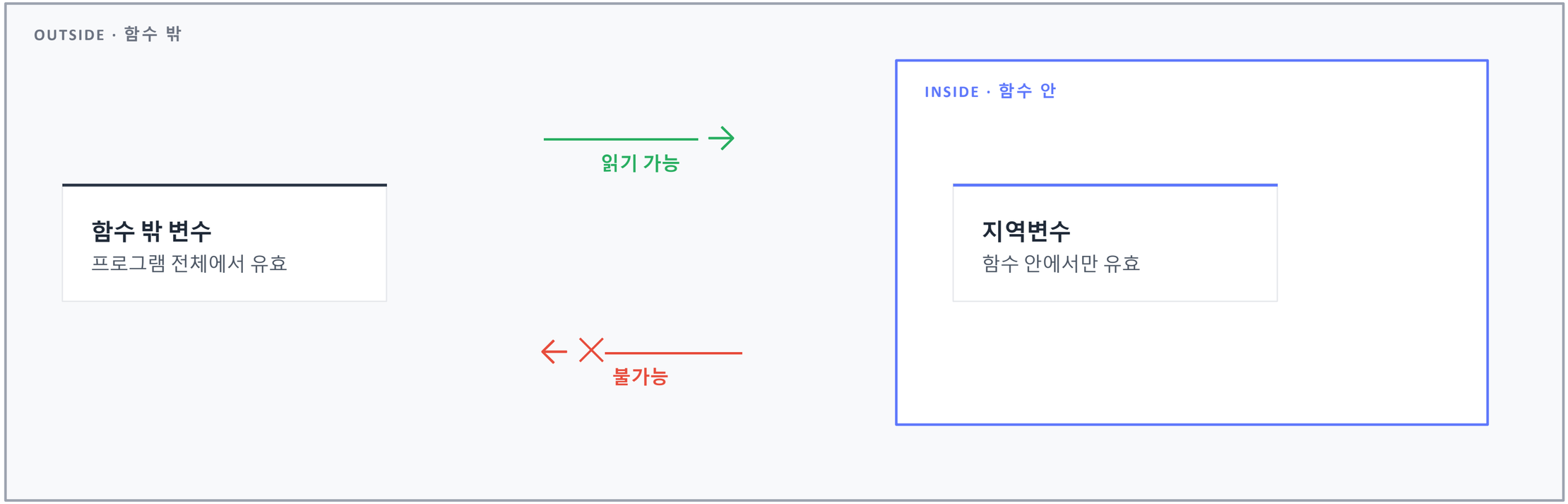
03 · 내보내기

밖에서 쓰려면 return

함수 안 계산 결과를
밖에서 쓰려면
return으로 돌려주면 된다.

함수 안 변수와 밖 변수

함수 안에서는 바깥 변수를 읽을 수 있지만, 바깥에서는 함수 안 지역변수를 읽을 수 없다.



함수 안에서 바깥 변수와 같은 이름으로 새 변수를 만들면 **바깥 변수를 바꾸는 게 아니라 함수 안에 별개의 지역변수**를 만드는 것.

함수는 깔끔한 통로로만 소통

함수에 필요한 값은 매개변수로 받고 결과는 return으로 내보낸다.

INPUT · 입력

매개변수로 받기

함수가 일하는 데 필요한 값은
매개변수로 받는다.

OUTPUT · 출력

return으로 내보내기

결과는 return으로 내보낸다.
분명한 창구로만 소통.

지역변수의 값의 수명

지역변수는 함수가 호출될 때 생겨나 함수가 끝날 때 사라지고, 다시 부르면 새로 만들어진다.

STEP 1 · 호출될 때

지역변수가 새로 생겨남

함수가 호출되는 순간 지역변수가 생긴다. 함수 안 코드가 실행되며 결과를 변수에 담는다.

STEP 2 · 함수 끝날 때

지역변수가 사라짐

함수의 실행이 끝나는 순간 지역변수들은 모두 사라진다. 함수가 끝난 뒤에는 찾을 수 없다.

STEP 3 · 다시 부를 때

완전히 새로 만들어짐

함수를 다시 호출하면 지역변수는 완전히 새로 만들어진다. 이전 호출의 흔적이 남지 않는다.

수명이 만드는 독립성

매번 지역변수가 새로 태어나 호출끼리 영향을 주지 않는다.

같은 입력에 같은 결과를 내는
예측 가능성의 바탕.

지역변수가 함수가 끝나도 사라지지 않고 값이 남는다면,
두 번째 호출이 첫 번째 호출의 값에 영향을 받아 예측하기 어려워질 것.
지역변수는 매번 깨끗이 새로 시작하니, 함수는 **언제 불러도 같은 입력에 같은 결과**를 낸다.

매개변수도 지역변수다

매개변수도 지역변수의 성질을 그대로 갖고 있다.

구분	매개변수	안에서 만든 변수
유효 범위	**함수 안**	**함수 안**
값 채움	호출 시 인자로부터	함수 안 코드에서
함수 끝나면	**사라짐**	**사라짐**

> 차이는 단 하나 — 매개변수는 **호출될 때 인자로부터 값을 받아 만들어진다**는 점.

함수 안 변수는 모두 지역변수

인자로 채워지는 매개변수와 직접 만드는 지역변수, 둘 다 같은 지역변수다.

01 · 매개변수

인자로 값을 받음

함수 안에서만 살고
함수가 끝나면 사라진다.

02 · 일반 지역변수

함수 안에서 직접 만듦

함수 안에서만 살고
함수가 끝나면 사라진다.

같은 이름 다른 변수

두 x는 이름만 같을 뿐 별개의 변수다.

CODE

```
x = 10
def change():
    x = 20   # 함수 안 x (별개)
change()
print(x)   # 출력: 10
```

함수 안의 x는 20, 함수 밖의 x는 여전히 10. 두 x는 **이름만 같을 뿐 별개**.

별개라서 충돌하지 않는다

함수 안에서 변수를 만들면 항상 그 함수의 지역변수가 된다.

안과 밖이 별개라
함수 안에서 원하는 이름을 마음껏 사용.

만약 함수 안에서 변수를 만들 때마다 밖의 같은 이름 변수를 덮어쓴다면
밖에 어떤 변수가 있는지 일일이 신경 써야 할 것.
여러 사람이 만든 함수를 한 프로그램에 모아도 **변수 이름이 충돌하지 않는다**.

posco

실습 2. 지역변수 관찰하기

함수 안에서 만든 지역변수가 함수 밖에서는 보이지 않음을 확인

단계

① 함수 안에서 새 변수를 만들어 사용
② 함수를 호출해 함수 안에서는 그 변수가 동작하는지 확인
③ 함수 밖에서 같은 변수를 출력해 오류가 나는지 확인
④ 함수 안 변수는 함수가 끝나면 사라지는 지역변수임을 정리

예상 결과 함수 안: 30 출력 / 함수 밖에서 접근하면 NameError

posco

실습 2. 지역변수 관찰하기 정답

CODE · 정답

```
def calc():
    result = 10 + 20    # result는 지역변수
    print(result)       # 30 (함수 안에서는 사용 가능)
calc()
# print(result)   # NameError: 함수 밖에서는 지역변수를 못 봄
```

posco

03

함수 설계와 조합

입력-처리-출력 · 함수 조합 · docstring · 체크리스트 · 파이프라인

함수를 나누면 좋은 이유

각 함수가 한 작업만 해 이해·수정·재사용이 쉬워진다.

01 · 이해 쉬움

각 함수가 한 작업만 해 이름으로 파악

복잡한 작업을 한 함수에 다 넣으면 거대한 덩어리.
작게 나누면 이름만 봐도 무슨 일을 하는지 안다.

02 · 수정 쉬움

바꿀 부분의 함수만 고치면 됨

계산 방식만 바꾸고 싶으면
계산 함수만 고치면 된다.
출력·정리 함수는 그대로.

03 · 재사용 쉬움

다른 작업에 가져다 쓰기

출력 함수가 따로 있으면
다른 작업에서도
그 출력 함수를 가져다 쓸 수 있다.

큰 문제를 작은 함수로

사람도 복잡한 일을 단계별로 나눠 하나씩 처리하듯 함수도 마찬가지다.

큰 작업을 작은 작업들로 나누고
각각을 함수로 만든 뒤 조합한다.

작은 함수 하나하나는 단순해서 만들기 쉽고, 엮으면 복잡한 일도 해낸다.

데이터 분석은 보통 **불러오기 → 거르기 → 통계 → 정리**의 여러 단계로 이루어진다.
각 단계를 함수로 나눠 두면 단계별로 점검·수정할 수 있어 체계적이고 안정적이다.

입력-처리-출력 사고법

함수를 만들기 전에 무엇을 입력받고, 처리하고, 출력할지 먼저 정한다.

함수	입력	처리	출력
평균 구하기	숫자 목록	합을 나눔	**평균값**
온도 판정	온도, 기준	비교	**정상 / 위험**
최댓값 찾기	숫자 목록	탐색	**최댓값**

입력은 매개변수로, 처리는 함수 안 코드로, 출력은 return으로 옮기면 함수의 뼈대가 완성된다.

함수 만들기 전 세 가지를 먼저

입력은 매개변수, 처리는 함수 안 코드, 출력은 return.

막연히 코드부터 짜지 말고
세 가지를 먼저 정하기.

막연히 "함수를 만들어야지" 하고 코드부터 짜는 것보다
입력-처리-출력을 먼저 정하고 시작하면 헤매지 않는다.
함수를 만들기 전에 세 가지를 **종이에 먼저 적어 보는 습관**이 좋다.

함수 하나에 기능 하나

한 함수는 하나의 명확한 기능만 담당.

하는 일을 한 문장으로
말할 수 있으면 좋은 함수.

데이터를 정리하는 함수는 정리만, 계산하는 함수는 계산만, 출력하는 함수는 출력만.
이렇게 하면 **이름 짓기가 쉽고, 수정이 쉽고, 재사용이 쉬워진다**.

posco

그리고가 여러 번이면 나누기

"이 함수가 하는 일을 한 문장으로 말할 수 있는가?"를 물어본다.

좋은 함수

한 문장으로
깔끔히 설명됨

"센서값의 평균을 구한다"
처럼 깔끔하면 좋은 함수.

나누기 신호

"그리고"가
여러 번 나옴

평균을 구하고, 비교하고,
출력하고, 저장하고...
그리고가 늘면 나눠야 한다.

균형

너무 잘게도
좋지 않다

하나의 의미 있는
기능 단위로 나누는
균형이 필요하다.

함수를 조합하는 두 방식

여러 함수를 엮어 전체 작업을 완성하는 것 — 이것이 함수 조합.

01 · 순서대로 호출

함수를 차례로 부르며 결과를 넘김

정리 함수 → 계산 함수 → 출력 함수.
한 함수의 결과를 변수에 담아
다음 함수에 넘긴다.

02 · 안에서 호출

큰 함수가 작은 함수들을 불러 씀

보고서 함수 안에서
평균 함수와 판정 함수를
불러 쓰는 식.

posco

함수 조합 예시

한 함수의 출력이 다음 함수의 입력과 맞아떨어져야 한다.

CODE

```
def get_average(values):
    return sum(values) / len(values)
avg = get_average([75, 88, 91])
print(avg)
```

get_average로 평균을 구해 avg에 담고, 그 avg를 다음 함수에 넘기면 **작업이 사슬처럼 이어진다**.

작은 함수들의 조합이 프로그래밍의 힘

작고 단순한 함수만 잘 만들면 엮어서 복잡한 작업을 완성한다.

큰 문제를 작은 함수로 나누고
다시 조합해 큰 작업으로.

우리는 **작고 단순한 함수**만 잘 만들면 된다.
그것들을 엮으면 아무리 복잡한 작업도 단계별로 해낼 수 있다.
복잡한 데이터 분석도 결국은 작은 함수들의 조합으로 이루어진다.

한 함수의 결과를 다른 함수에

변수에 담아 넘기는 방법과, 변수 없이 바로 넘기는 방법이 있다.

CODE

```
# (1) 변수를 거쳐 넘기는 방법 – 중간값이 보여 읽기 쉬움
avg = get_average(temps)
result = judge(avg)
# (2) 변수 없이 바로 넘기는 방법 – 코드가 짧음
result = judge(get_average(temps))
```

위는 중간값을 변수에 담아 다른 곳에도 쓰기 좋고, 아래는 한 줄로 짧게 표현된다.

입문 단계에는 변수를 거치기

두 방법을 비교하고, 입문 단계에서는 변수를 거치는 방법을 권장한다.

01 · 변수 거치기 (권장)

중간값이 보여 읽기 쉽고 다른 데도 사용

중간 결과에 이름이 붙으니
무슨 일이 일어나는지 분명하다.

02 · 바로 넘기기

코드가 짧지만 여러 겹이면 읽기 어려움

단순히 결과를 한 번 넘기기만 하면
되고 짧게 하고 싶을 때.

실습 3. 처리 흐름 만들기

작은 함수들을 정의하고 한 함수의 결과를 다른 함수에 이어 전달하기

단계

① 값을 받아 계산해 return하는 함수를 정의
② 계산 결과를 받아 판정해 출력하는 함수를 정의
③ 첫 함수의 반환값을 변수에 담아 둘째 함수에 전달
④ 실행해 입력-처리-출력 흐름이 이어지는지 확인

예상 결과 평균 85.0 → 정상

posco

실습 3. 처리 흐름 만들기 정답

CODE · 정답

```
def average(a, b):
    return (a + b) / 2
def judge(value):
    if value > 90:
        print("경고")
    else:
        print("정상")
avg = average(80, 90)   # 처리 결과
judge(avg)              # 이어서 판정 → 정상
```

함수에 설명 달기 docstring

함수를 정의한 바로 다음 줄에 따옴표 세 개로 설명문을 적는다.

CODE

```
def get_average(values):
    """숫자 목록을 받아 평균을 돌려준다."""
    return sum(values) / len(values)
```

함수가 **무엇을 하는지, 어떤 값을 입력받는지, 무엇을 돌려주는지**를 적는다. 짧으면 한 줄, 길면 여러 줄.

docstring의 세 가지 이점

쓰는 사람에게 사용법을 알려주고 미래의 나에게도 도움이 된다.

01 · 쓰는 사람

사용법을 알려준다

이 함수가 무엇을 받고
무엇을 돌려주는지
한눈에 알 수 있다.

02 · 미래의 나

기억을 되살린다

몇 주 뒤 다시 본 미래의 나에게도 친절한 도움말이 된다.

03 · 협업

사용 설명서가 된다

여러 사람이 협업할 때
함수의 사용 설명서
역할을 한다.

좋은 함수의 조건 정리

좋은 함수는 다시 쓰기 쉽고 읽기 쉽고 고치기 쉽다.

항목	좋은 함수	아쉬운 함수
기능	한 가지 일	여러 일 섞임
이름	의미 명확	모호함
입출력	분명함	불명확
설명	docstring 있음	설명 없음

> **코드는 계속 읽고 고치고 재사용되기 때문에** 이 세 가지 쉬움이 함수의 품질을 결정한다.

posco

세 가지 쉬움이 핵심 정신

다시 쓰기 쉽고 읽기 쉽고 고치기 쉽다.

01 · REUSE

다시 쓰기 쉽다

02 · READ

읽기 쉽다

03 · CHANGE

고치기 쉽다

처음부터 완벽한 함수를 만들 필요는 없다. **일단 동작하게 만들고 조금씩 다듬어 간다**.

함수 작성 시 흔한 실수

대부분 에러 메시지가 원인을 알려 준다.

실수	증상	확인할 점
괄호 누락	**실행 안 됨**	호출에 괄호 있는지
return 누락	**결과가 None**	return 확인
인자 수 불일치	**인자 에러**	개수 맞추기
정의 전 호출	**이름 에러**	정의 먼저

NameError → 정의 순서나 지역변수를 의심

에러 메시지가 원인을 알려준다

실수 목록을 외우기보다 에러 메시지를 읽는 습관을 들인다.

에러는 함수를 배우는 과정의 **자연스러운 일부**다.

에러 메시지를 두려워하지 말고 **그것이 주는 단서를 읽는 습관**을 들이면
실수를 빠르게 바로잡을 수 있다.
실수하고 고치고 배우는 과정을 반복하면서 실력이 는다.

함수 설계 체크리스트

함수를 만든 뒤 네 가지를 스스로에게 물어본다.

점검 항목	질문
01 · 기능	한 문장으로 설명되는가
02 · 이름	이름만 봐도 아는가
03 · 입출력	입력과 출력이 분명한가
04 · 동작	여러 입력으로 테스트했는가

처음엔 의식적으로 나중엔 자동으로

처음에는 질문을 하나씩 짚고 익숙해지면 자동으로 점검한다.

처음에는

의식적으로 하나씩 짚기

운전을 배울 때 거울 보기를
의식하다 나중엔 몸에 배는 것과
같다.

함수 중요도에 따라

점검 강도 조절

간단한 함수는 이름만 명확해도 충분.
중요한 함수는 모든 항목을
꼼꼼히 점검할 가치가 있다.

데이터 처리 파이프라인 맛보기

원본 → 거르기 → 통계 → 정리 → 결과로 데이터가 단계별로 흐른다.

STEP 1

원본 데이터

측정해 들어온 그대로의 원본 데이터.

입력

→

STEP 2

이상값 제거

거르기 함수가 이상한 값을 걸러낸다.

거르기

→

STEP 3

평균 계산

통계 함수가 평균 같은 값을 계산한다.

통계

→

STEP 4

결과 포맷

정리 함수가 결과를 보기 좋게 정리한다.

정리

→

STEP 5

최종 결과

사용자에게 전달되는 최종 결과물.

결과

각 단계가 독립된 함수라서, 한 단계를 점검하거나 바꿀 때 다른 단계에 영향을 주지 않는다.

함수가 분석의 기본 부품

함수들을 이어 데이터를 단계별로 처리하는 것이 분석의 기본 구조다.

지금 배운 함수가 앞으로 모든 데이터 작업의 부품이 된다.

지금 단계에서 파이프라인을 완벽히 만들 필요는 없다.
다만 **함수들을 이어 데이터를 단계별로 처리한다는 큰 그림**을 알아 두면
앞으로 데이터 분석을 배울 때 이 구조가 계속 나타나는 걸 보게 된다.

실습 4. 센서 분석 함수 세트 만들기

매개변수·기본값·반환값·함수 조합을 모두 써서 목록에서 상태까지 구하기

단계

① 센서값 목록을 받아 평균을 return하는 함수를 정의
② 평균과 기준값(기본값 있음)을 받아 상태를 판정해 return하는 함수를 정의
③ 두 함수를 순서대로 연결해 목록에서 상태까지 구하기
④ 실행해 흐름과 결과를 확인

예상 결과 85.0 정상

posco

실습 4. 센서 분석 함수 세트 만들기 정답

CODE · 정답

```
def average(values):
    return sum(values) / len(values)
def status(avg, limit=90):
    if avg > limit:
        return "경고"
    else:
        return "정상"
avg = average([78, 92, 85])
print(avg, status(avg))   # 85.0 정상
```